

PRODUCT AND CATEGORY MANAGEMENT

Project Title: Shop with Nykaa

1. Introduction

The Shop with Nykaa Product and Category Management System is designed to manage product and category information efficiently in a beauty and personal care shopping database. The system uses related tables to store and organize product and category details.

Primary key and foreign key constraints are used to maintain data integrity and establish relationships between the entities.

SQL operations such as INSERT, UPDATE, and DELETE are performed to manage product information. Category-wise reports can be generated using SQL to analyze products, prices, and stock information.

2. Objectives

- Design tables for Product and Category entities.
- Define primary key and foreign key relationships.
- Store product name, category, price, and stock details.
- Perform product insertion, updating, and deletion.
- Generate category-wise product reports.
- Maintain data integrity using constraints.

3. Relationship

The Shop with Nykaa database contains relationships between customers, products, categories, suppliers, orders, order details, payments, inventory and admins.

CUSTOMER (1) ————— (M) ORDER

SUPPLIER (1) ————— (M) PRODUCT

CATEGORY (1) ————— (M) PRODUCT

PRODUCT (1) ————— (M) ORDER DETAILS

ORDER (1) ————— (M) ORDER DETAILS

ORDER (1) ————— (1) PAYMENT

PRODUCT (1) ————— (1) INVENTORY

ADMIN (1) ————— (M) PRODUCT

ADMIN (1) ————— (M) ORDER

The Category–Product relationship is **1:M**, meaning one category can contain many products, while each product belongs to one category.

4. Table Design

4.1 Category Table

Attribute	Data Type	Constraint
Category_ID	NUMBER(10)	Primary Key
Category_Name	VARCHAR2(50)	NOT NULL, UNIQUE
Category_Description	VARCHAR2(200)	—
Status	VARCHAR2(20)	CHECK

4.2 Product Table

Attribute	Data Type	Constraint
Product_ID	NUMBER(10)	Primary Key
Product_Name	VARCHAR2(100)	NOT NULL
Category_ID	NUMBER(10)	Foreign Key
Supplier_ID	NUMBER(10)	Foreign Key
Price	NUMBER(10,2)	NOT NULL
Weight	VARCHAR2(20)	—
Ingredients	VARCHAR2(500)	—
Skin_Type	VARCHAR2(30)	—
Brand	VARCHAR2(50)	—
Expiry_Date	DATE	—

5. Table Creation

5.1 Create Category Table

```
SQL> CREATE TABLE Category (  
2     Category_ID NUMBER PRIMARY KEY,  
3     Category_Name VARCHAR2(50) NOT NULL,  
4     Description VARCHAR2(200),  
5     Status VARCHAR2(20)  
6 );
```

Table created.

5.2 Create Product Table

```
SQL> CREATE TABLE Product (  
2     Product_ID NUMBER PRIMARY KEY,  
3     Product_Name VARCHAR2(100) NOT NULL,  
4     Brand_ID NUMBER,  
5     Category_ID NUMBER,  
6     Price NUMBER(10,2),  
7     Stock NUMBER,  
8     Description VARCHAR2(200),  
9     Skin_Type VARCHAR2(50),  
10    Rating NUMBER(2,1),  
11    Expiry_Date DATE,  
12    CONSTRAINT fk_product_category  
13    FOREIGN KEY (Category_ID)  
14    REFERENCES Category(Category_ID)  
15 );
```

Table created.

6 Data Insertion

6.1 Category Records

```
SQL> INSERT INTO Category VALUES (101, 'Skincare', 'Skincare products', 'Active');  
1 row created.  
  
SQL> INSERT INTO Category VALUES (102, 'Makeup', 'Makeup and cosmetics', 'Active');  
1 row created.  
  
SQL> INSERT INTO Category VALUES (103, 'Haircare', 'Hair care products', 'Active');  
1 row created.  
  
SQL> INSERT INTO Category VALUES (104, 'Fragrance', 'Perfumes and fragrances', 'Active');  
1 row created.  
  
SQL> INSERT INTO Category VALUES (105, 'Bodycare', 'Body care products', 'Active');  
1 row created.  
  
SQL> INSERT INTO Category VALUES (106, 'Bath and Shower', 'Bath and shower products', 'Active');  
1 row created.  
  
SQL> INSERT INTO Category VALUES (107, 'Nails', 'Nail care products', 'Active');  
1 row created.  
  
SQL> INSERT INTO Category VALUES (108, 'Lip Care', 'Lip care products', 'Active');  
1 row created.  
  
SQL> INSERT INTO Category VALUES (109, 'Sun Care', 'Sunscreen and sun protection products', 'Active');  
1 row created.  
  
SQL> INSERT INTO Category VALUES (110, 'Men Grooming', 'Grooming products for men', 'Active');  
1 row created.
```

6.2 Product Records

```
SQL> Insert into Product values
  2 (1001, 'Vitamin C Face Serum', 101, 101, 599.00, 50,
  3 'Vitamin C serum for glowing skin', 'All Skin Types', 4.5, DATE '2028-12-31');

1 row created.

SQL>
SQL> Insert into Product values
  2 (1002, 'Matte Lipstick', 102, 102, 399.00, 55,
  3 'Long lasting matte lipstick', 'All Skin Types', 4.5, DATE '2029-01-31');

1 row created.

SQL>
SQL> Insert into Product values
  2 (1003, 'Anti Dandruff Shampoo', 103, 103, 349.00, 50,
  3 'Shampoo for dandruff control', 'All Hair Types', 4.3, DATE '2029-01-31');

1 row created.

SQL>
SQL> INSERT INTO Product VALUES
  2 (1004, 'Floral Eau De Parfum', 104, 104, 899.00, 30,
  3 'Fresh floral fragrance', 'All Skin Types', 4.5, DATE '2029-01-31');

1 row created.

SQL>
SQL> INSERT INTO Product VALUES
  2 (1005, 'Body Lotion', 105, 105, 349.00, 60,
  3 'Moisturizing body lotion', 'Dry Skin', 4.4, DATE '2029-01-31');

1 row created.

SQL>
SQL> INSERT INTO Product VALUES
  2 (1006, 'Lavender Shower Gel', 106, 106, 299.00, 50,
  3 'Relaxing lavender shower gel', 'All Skin Types', 4.5, DATE '2029-01-31');

1 row created.

SQL>
SQL> INSERT INTO Product VALUES
  2 (1007, 'Red Nail Polish', 107, 107, 199.00, 60,
  3 'Glossy red nail polish', 'All Skin Types', 4.4, DATE '2029-01-31');

1 row created.

SQL>
SQL> INSERT INTO Product VALUES
  2 (1008, 'Lip Balm', 108, 108, 149.00, 80,
  3 'Moisturizing lip balm', 'All Skin Types', 4.5, DATE '2029-01-31');

1 row created.

SQL>
SQL> INSERT INTO Product VALUES
  2 (1009, 'SPF 50 Sunscreen', 109, 109, 599.00, 60,
  3 'High protection sunscreen', 'All Skin Types', 4.6, DATE '2029-01-31');

1 row created.

SQL>
SQL> INSERT INTO Product VALUES
  2 (1010, 'Beard Oil', 110, 110, 399.00, 50,
  3 'Nourishing beard oil', 'All Skin Types', 4.5, DATE '2029-01-31');

1 row created.
```

7 . Data Retrieval

7.1 Display Category Records

```
SQL> SELECT * FROM Category;

CATEGORY_ID CATEGORY_NAME
-----
DESCRIPTION
-----
STATUS
-----
101 Skincare
Skincare products
Active

102 Makeup
Makeup and cosmetics
Active

CATEGORY_ID CATEGORY_NAME
-----
DESCRIPTION
-----
STATUS
-----
103 Haircare
Hair care products
Active

104 Fragrance
Perfumes and fragrances

CATEGORY_ID CATEGORY_NAME
-----
DESCRIPTION
-----
STATUS
-----
Active

105 Bodycare
Body care products
Active

106 Bath and Shower

CATEGORY_ID CATEGORY_NAME
-----
DESCRIPTION
-----
STATUS
-----
Bath and shower products
Active

107 Nails
Nail care products
Active

CATEGORY_ID CATEGORY_NAME
-----
DESCRIPTION
-----
STATUS
-----
108 Lip Care
Lip care products
Active

109 Sun Care
Sunscreen and sun protection products
Active

CATEGORY_ID CATEGORY_NAME
-----
DESCRIPTION
-----
STATUS
-----
110 Men Grooming
Grooming products for men
Active

10 rows selected.

SQL> commit;
```

7.2 Display Product Records

[illegible]

8. Product Update

```
SQL> UPDATE Product
  2 SET Price = 649.00,
  3     Stock = 45
  4 WHERE Product_ID = 1001;

1 row updated.

SQL> UPDATE Product
  2 SET Price = 449.00,
  3     Stock = 60
  4 WHERE Product_ID = 1002;

1 row updated.
```

8.1 Product Deletion

```
SQL> DELETE FROM Product
  2 WHERE Product_ID = 1009;

1 row deleted.

SQL> DELETE FROM Product
  2 WHERE Product_ID = 1010;

1 row deleted.
```

9. Category-wise Product Reports

```
SQL> SELECT
  2     c.Category_Name,
  3     COUNT(p.Product_ID) AS Total_Products,
  4     SUM(p.Stock) AS Total_Stock
  5 FROM Category c
  6 LEFT JOIN Product p
  7 ON c.Category_ID = p.Category_ID
  8 GROUP BY c.Category_ID, c.Category_Name
  9 ORDER BY c.Category_ID;

CATEGORY_NAME          TOTAL_PRODUCTS  TOTAL_STOCK
-----
Skincare                1              45
Makeup                  1              60
Haircare                1              50
Fragrance               1              30
Bodycare                1              60
Bath and Shower         1              50
Nails                   1              60
Lip Care                1              80
Sun Care                0
Men Grooming            0

10 rows selected.
```

10. Product and Category Report

SQL> SELECT
2 p.Product_ID
3 p.Product_Name
4 c.Category_ID
5 c.Category_Name
6 p.Price
7 p.Weight
8 FROM Products p
9 JOIN Category c
10 ON c.Category_ID = p.Category_ID
11 ORDER BY c.Category_ID

PRODUCT_ID	PRODUCT_NAME	PRICE
1001	Virtuality T-Shirt Green 100 Grams	4.99
1002	Matrix T-Shirt 100 Grams	4.99
1003	Wall Mounted Shredder 150 Pounds	249
1004	Floral Bath Soak 100 Pounds	249
1005	Body Lotion 100 Pounds	249
1006	Excessive Shaver Gel 100 Gals and Shaver	249
1007	Hot Nail Polish 100 Puffs	1.99
1008	Lip Balm 100 Lip Care	1.99

5 rows selected.
SQL> |

11. Expected Category-wise Report

Category	Total Products	Total Stock
Makeup	1	60
Skin Care	1	40
Hair Care	1	30
Fragrance	1	50
Bath & Body	1	25
Face Care	1	30
Lip Care	1	45
Nail Care	1	40
Sun Care	1	35
Wellness	1	20

12. Conclusion

The Shop with Nykaa Product and Category Management System successfully represents the relationships between customers, suppliers, categories, products, orders, order details, payments, inventory and admins.

Primary key and foreign key constraints help maintain data integrity and establish relationships between the tables.

INSERT, UPDATE and DELETE operations support effective product and category management. SQL reports provide useful information about products, prices and stock, while PL/SQL programs can demonstrate procedural database programming for category-wise report generation and product price updates.